C++习题课（7，9）

习题七

A28

定义一个不受计算机字长限制的整数类INT,要求INT与INT以及INT与c++基本数据类型int之间能进行+、-、*、/和=运算，并且能够通过count输出INT类型的值。

```
#include <iostream>
#include <cstring>
using namespace std;

class INT
{
private:
    char *p_buf; //从低位到高位存储整型数
    int buf_len; //p_buf所占空间大小（为10的倍数）
    int sign;    //整型数的符号
    void Add(const INT &i);
    void Minus(const INT &i);
    INT(char *p, int len, int s = 1);

public:
    INT();
    INT(int i);
    INT(char *num);
    INT(const INT &i);
    ~INT();
    INT &operator=(const INT &i);
    INT &operator+=(const INT &i);
    INT &operator-=(const INT &i);
    friend INT operator+(const INT &i1, const INT &i2);
    friend INT operator-(const INT &i1, const INT &i2);
    friend INT operator*(const INT &i1, const INT &i2);
    friend INT operator/(const INT &i1, const INT &i2);
    friend ostream &operator<<(ostream &out, const INT &i);
};

INT::INT()
{
    sign = 1;
    buf_len = 10;
    p_buf = new char[buf_len + 1];
    p_buf[0] = '0';
    p_buf[1] = '\0';
}

```

```
INT::INT(int i)
{
    if (i >= 0)
        sign = 1;
    else
    {
        sign = -1;
        i = -i;
    }

    int j = i, k = 0;
    do
    {
        k++;
        j /= 10;
    } while (j != 0);

    buf_len = (k / 10 + 1) * 10;
    p_buf = new char[buf_len + 1];
    j = 0;

    do
    {
        k = i % 10;
        p_buf[j++] = k + '0';
        i /= 10;
    } while (i != 0);
    p_buf[j] = '\0';
}

INT::INT(char *num)
{
    if (*num == '-')
    {
        sign = -1;
        num++;
    }
    else if (*num == '+')
    {
        sign = 1;
        num++;
    }
    else
        sign = 1;

    buf_len = (strlen(num) / 10 + 1) * 10;
    p_buf = new char[buf_len + 1];

    for (int i = 0, j = strlen(num) - 1; j >= 0; i++, j--)
```

```
        p_buf[i] = num[j];
    p_buf[strlen(num)] = '\0';
}

INT::INT(const INT &i)
{
    sign = i.sign;
    buf_len = i.buf_len;
    p_buf = new char[buf_len + 1];
    strcpy(p_buf, i.p_buf);
}

INT::INT(char *p, int len, int s)
{
    p_buf = p;
    buf_len = len;
    sign = s;
}

INT::~INT()
{
    delete[] p_buf;
    p_buf = NULL;
    buf_len = 0;
}

INT &INT::operator=(const INT &i)
{
    sign = i.sign;
    if (buf_len < i.buf_len)
    {
        delete[] p_buf;
        buf_len = i.buf_len;
        p_buf = new char[buf_len + 1];
    }
    strcpy(p_buf, i.p_buf);
    return *this;
}

void INT::Add(const INT &i)
{ //把i加到*this中
    int len1 = strlen(p_buf);
    int len2 = strlen(i.p_buf);
    int buf_len1 = (((len1 > len2 ? len1 : len2) + 1) / 10 + 1) * 10;
    if (buf_len1 > buf_len)
    {
        char *p_buf1 = new char[buf_len1 + 1];
        strcpy(p_buf1, p_buf);
        delete[] p_buf;
```

```
        p_buf = p_buf1;
        buf_len = buf_len1;
    }

    char *p1, *p2; // p1指向长的数，p2指向短的数
    if (len1 >= len2)
    {
        p1 = p_buf;
        p2 = i.p_buf;
    }
    else
    {
        p1 = i.p_buf;
        p2 = p_buf;
    }

    int carry = 0, sum;
    char *p = p_buf;
    //处理公共长度部分
    while (*p2 != '\0')
    {
        sum = (*p1 - '0') + (*p2 - '0') + carry;
        if (sum >= 10)
        {
            carry = 1;
            *p = (sum - 10) + '0';
        }
        else
        {
            carry = 0;
            *p = sum + '0';
        }
        p1++;
        p2++;
        p++;
    }
    //处理较大整数的剩余部分
    while (*p1 != '\0')
    {
        if (carry == 0)
            *p = *p1;
        else
        {
            sum = (*p1 - '0') + carry;
            if (sum >= 10)
            {
                carry = 1;
                *p = (sum - 10) + '0';
            }
```

```
            else
            {
                carry = 0;
                *p = sum + '0';
            }
        }
        p1++;
        p++;
    }
    if (carry != 0)
    { //最后检查是否还有进位，若有就放入和的最高位
        *p = '1';
        p++;
    }
    *p = '\0';
}

void INT::Minus(const INT &i)
{ //从*this中减去i
    int len1 = strlen(p_buf);
    int len2 = strlen(i.p_buf);
    int buf_len1 = ((len1 > len2 ? len1 : len2) / 10 + 1) * 10;
    if (buf_len1 > buf_len)
    {
        char *p_buf1 = new char[buf_len1 + 1];
        strcpy(p_buf1, p_buf);
        delete[] p_buf;
        p_buf = p_buf1;
        buf_len = buf_len1;
    }

    char *p1 = NULL, *p2;

    if (len1 > len2)
        p1 = p_buf;
    else if (len1 < len2)
        p1 = i.p_buf;
    else
    {
        int j;
        for (j = len1 - 1; j >= 0; j--)
        {
            if (p_buf[j] > i.p_buf[j])
            {
                p1 = p_buf;
                break;
            }
            else if (p_buf[j] < i.p_buf[j])
            {
```

```
                p1 = i.p_buf;
                break;
            }
        }
        if (j < 0)
        { //如果两个数完全相同
            sign = 1;
            strcpy(p_buf, "0");
            return;
        }
    }
    if (p1 == p_buf)
    {
        p2 = i.p_buf;
    }
    else
    {
        p2 = p_buf;
        sign = i.sign;
    }
    int carry = 0, diff;
    char *p = p_buf;
    //公有长度部分相减
    while (*p2 != '\0')
    {
        diff = *p1 - *p2 - carry;
        if (diff < 0)
        {
            *p = (diff + 10) + '0';
            carry = 1;
        }
        else
        {
            *p = diff + '0';
            carry = 0;
        }
        p1++;
        p2++;
        p++;
    }
    //将被减数中未参与相减的高位也计算到差中
    while (*p1 != '\0')
    {
        if (carry == 0)
            *p = *p1;
        else
        {
            diff = (*p1 - '0') - carry;
            if (diff < 0)
```

```
            {
                *p = (diff + 10) + '0';
                carry = 1;
            }
            else
            {
                *p = diff + '0';
                carry = 0;
            }
        }
        p1++;
        p++;
    }
    p--;
    while (p != p_buf && *p == '0')
        p--; //去掉高位的0
    *(p + 1) = '\0';
}

INT &INT::operator+=(const INT &i)
{
    if (sign == i.sign) //如果两个数符号相同，绝对值相加
        Add(i);
    else //如果两个数符号相反，绝对值相减
        Minus(i);
    return *this;
}

INT operator+(const INT &i1, const INT &i2)
{
    INT temp(i1);
    temp += i2;
    return temp;
}

INT &INT::operator-=(const INT &i)
{
    INT *p = (INT *)&i;
    p->sign = -p->sign; //把i变成-i
    *this += *p;        // *this += -i
    p->sign = -p->sign; //把i还原
    return *this;
}

INT operator-(const INT &i1, const INT &i2)
{
    INT temp(i1);
    temp -= i2;
    return temp;
```

```
}

INT operator*(const INT &i1, const INT &i2)
{
    int len1 = strlen(i1.p_buf);
    int len2 = strlen(i2.p_buf);
    int buf_len = ((len1 + len2) / 10 + 1) * 10;
    char *p_buf = new char[buf_len + 1];
    INT product, temp(p_buf, buf_len, 1);
    //用一个乘数(i1)的每一位与另一个乘数(i2)相乘,
    //结果(temp)向左移相应的位数后加到乘积（product）中去
    for (int j = 0; j < len1; j++)
    {
        int carry = 0, mul, n;
        for (n = 0; n < j; n++)
            p_buf[n] = '0';
        for (int k = 0; k < len2; k++)
        {
            mul = (i1.p_buf[j] - '0') * (i2.p_buf[k] - '0') + carry;
            carry = mul / 10;
            p_buf[n++] = mul % 10 + '0';
        }
        if (carry != 0)
            p_buf[n++] = carry + '0';
        p_buf[n] = '\0';
        product += temp; //将每一位与另一个乘数相乘的结果累加就可以得到两数积
    }
    if (i1.sign == i2.sign) //最后统一处理积的符号
        product.sign = 1;
    else
        product.sign = -1;
    return product;
}

INT operator/(const INT &i1, const INT &i2)
{
    INT div1(i1), div2(i2), quotient;
    div1.sign = 1;
    div2.sign = 1;

    //不断地从“被除数”中减去“除数”，直到“被除数”小于0，这时，减去的“除数”的个数即为“商”。
    div1 -= div2; //用被除数减去除数
    while (div1.sign > 0)
    {                   //直到被除数小于零
        quotient += 1; //每减一次除数，计数器加1
        div1 -= div2;  //用被除数减去除数
    }
    if (i1.sign == i2.sign) //最后统一处理商的符号
        quotient.sign = 1;
```

```
    else
        quotient.sign = -1;
    return quotient;
}

ostream &operator<<(ostream &out, const INT &i)
{
    if (i.sign < 0)
        out << '-';
    for (int j = strlen(i.p_buf) - 1; j >= 0; j--)
        out << i.p_buf[j];
    return out;
}
```

A29

设计一种解决例6-10中把存储块归还到堆空间的方法。（提示：可以在每次申请存储块时多申请一个能存储一个指针的空间，用该指针把各个存储块链接起来）

```
#include <cstring>
using namespace std;
const int NUM = 2;
class A
{ //...... //类A的已有成员说明。
public:
    static void *operator new(size_t size);
    static void operator delete(void *p);
    static void free_blocks(); //归还申请的所有存储块。
private:
    static A *p_free;     //用于指向A类对象的自由空间链表。
    A *next;              //用于实现自由空间结点的链接。
    static char *p_block; //用于指向由NUM个A类对象所组成的存储块所构成的链表。
};
A *A::p_free = NULL;
char *A::p_block = NULL;
void *A::operator new(size_t size)
{
    A *p;
    if (p_free == NULL)
    {
        char *q = new char[sizeof(char *) + size * NUM]; //申请一个块指针和NUM个A类对象所构成的堆空间。
        *(char **)q = p_block;                           //建立存储块链表。
        p_block = q;

        p_free = (A *)(q + sizeof(char *));          //跳过块指针。
        for (p = p_free; p != p_free + NUM - 1; p++) //建立自由结点链表。
            p->next = p + 1;
        p->next = NULL;
```

```
    }
    p = p_free; //为当前创建的对象分配空间。
    p_free = p_free->next;
    memset(p, 0, size);
    return p;
}
void A::operator delete(void *p)
{
    ((A *)p)->next = p_free;
    p_free = (A *)p;
}

void A::free_blocks() //归还申请的所有存储块。
{
    while (p_block != NULL)
    {
        char *p = p_block;
        p_block = *(char **)p_block;
        delete[] p;
    }
}
```

A31

完善例6-12中String类的功能，使其能够提高下面代码的效率：

```
String f()
{
  String t("1234");
  ......
  return t;

}
int main()
{
  String a = "abcd"
  ......
  a = f();
  ......
}
```

```
#include <iostream>
#include <cstdlib>
#include <cstring>
using namespace std;
class String
{
    char *str;
```

```
public:
    String(String &&s) //增加转移构造函数
    {
        str = s.str;
        s.str = NULL;
    }
    String &operator=(String &&x) //增加赋值操作符重载函数
    {
        if (&x == this)
            return *this;
        delete[] str;
        str = x.str;
        x.str = NULL;
        return *this;
    }
};
```

A1～A2

1）在C++中，protected类成员访问控制有什么作用?
C++中引进protected成员保护控制，缓解了数据封装与继承的矛盾。在基类中声明为protected的成员可以被派生类使用，但不能被基类的实例用户使用，这样能够对修改基类的内部实现所造成的影响范围（只影响子类）进行控制。protected成员保护控制的引进使得类有两种接口：与实例用户的接口和与派生类用户的接口。

2）在C++中，继承方式的作用是什么？public继承方式有什么特殊之处?
继承允许我们依据另一个类来定义一个类，这使得创建和维护一个应用程序变得更容易。这样做，也达到了重用代码功能和提高执行时间的效果。public继承方式使得基类的public成员可以被派生类的对象访问，它可以实现类之间的子类型关系。

3）在C++中，虚函数的作用是什么?
C++中的虚函数的作用主要是实现了多态的机制，关于多态，简而言之就是用父类型别的指针指向其子类的实例，然后通过父类的指针调用实际子类的成员函数。

4）在多继承中，什么情况下会出现二义性？怎样消除二义性?
在多继承中会出现两个问题：名冲突和重复继承。在多继承中，当多个基类中包含同名的成员时，它们在派生类中就会出现名冲突问题；在多继承中，如果直接基类有公共的基类，就会出现重复继承，这样，公共基类中的数据成员在多继承的派生类中就有多个拷贝。在C++中，解决名冲突的方法是用基类名受限；解决重复继承问题的手段是采用虚基类。

A7

利用习题6.8的第14题中的时间类Time，定义一个带时区的时间类ExtTime。除了构造函数和时间调整函数外，ExtTime的其它功能与Time类似。

```
#include <iostream>
using namespace std;

class Time
{
public:
```

```
    Time()
    {
        hour = minute = second = 0;
    }
    Time(int h, int m, int s)
    {
        hour = h;
        minute = m;
        second = s;
    }
    void set(int h, int m, int s)
    {
        hour = h;
        minute = m;
        second = s;
    }

    void increment()
    {
        if (second == 59)
        {
            if (minute == 59)
            {
                if (hour == 23)
                {
                    hour = 0;
                    minute = 0;
                    second = 0;
                }
                else
                {
                    hour++;
                    minute = 0;
                    second = 0;
                }
            }
            else
            {
                minute++;
                second = 0;
            }
        }
        else
            second++;
    }

    void display() const
    {
        cout << hour << ':' << minute << ':' << second;
```

```
    }
    bool equal(Time &other_time) const
    {
        if (hour == other_time.hour && minute == other_time.minute && second == other_time.second)
            return true;
        else
            return false;
    }
    bool less_than(Time &other_time) const
    {
        if (hour < other_time.hour ||
            hour == other_time.hour && minute < other_time.minute ||
            hour == other_time.hour && minute == other_time.minute && second < other_time.second)
            return true;
        else
            return false;
    }

protected:
    int hour;
    int minute;
    int second;
};

enum TimeZone
{
    W12 = -12,
    W11,
    W10,
    W9,
    W8,
    W7,
    W6,
    W5,
    W4,
    W3,
    W2,
    W1,
    GMT,
    E1,
    E2,
    E3,
    E4,
    E5,
    E6,
    E7,
    E8,
```

```
    E9,
    E10,
    E11,
    E12
};

class ExtTime : public Time
{
public:
    ExtTime() { tz = GMT; }
    ExtTime(int h, int m, int s, TimeZone t) : Time(h, m, s) { tz = t; }
    void set(int h, int m, int s, TimeZone t)
    {
        Time::set(h, m, s);
        tz = t;
    }
    void display() const
    {
        if (tz < 0)
            cout << "西" << -int(tz) << "区\t";
        else if (tz == 0)
            cout << "格林威治(子午线)标准时间  (GMT)\t";
        else
            cout << "东" << int(tz) << "区\t";
        Time::display();
    }
    bool equal(ExtTime &other_time) const
    {
        Time t((hour + other_time.tz - tz + 24) % 24, minute, second);
        return t.equal(other_time);
    }
    bool less_than(ExtTime &other_time) const
    {
        Time t((hour + other_time.tz - tz + 24) % 24, minute, second);
        return t.less_than(other_time);
    }

private:
    TimeZone tz;
};
```

A8

利用习题6.8的第18题中的LinearList类定义一个栈类。

```
#include <iostream>
using namespace std;

class LinearList
```

```
{
public:
    LinearList();
    ~LinearList();
    bool insert(int x, int pos);
    bool remove(int &x, int pos);
    int element(int pos) const;
    int search(int x) const;
    int length() const;

private:
    struct Node
    {
        int value;
        Node *next;
    };
    int count;
    Node *head;
};

class Stack : LinearList
{
public:
    bool push(int x);
    bool pop(int &x);
    LinearList::length;
};

bool Stack::push(int x)
{
    return insert(x, 0);
}

bool Stack::pop(int &x)
{
    return remove(x, 1);
}
```

B1

定义一个三维向量类，通过运算符重载“>”、“<”，比较标量的大小。

```
#include <cstring>
#include <iostream>
using namespace std;

class ThreeVector
{
public:
```

```
    ThreeVector(double xV, double yV, double zV)
    {
        x = xV;
        y = yV;
        z = zV;
    }
    ~ThreeVector();
    double Scalar()
    {
        return sqrt(x * x + y * y + z * z);
    }
    friend ostream &operator<<(ostream &output, ThreeVector &v);

private:
    double x, y, z;
};
bool operator>(ThreeVector v1, ThreeVector v2)
{
    return v1.Scalar() > v2.Scalar();
}
bool operator<(ThreeVector v1, ThreeVector v2)
{
    return v1.Scalar() < v2.Scalar();
}
ostream &operator<<(ostream &output, ThreeVector &v) //定义运算符“<<”重载函数
{
    output << "(" << v.x << "," << v.y << "," << v.z << ")";
    return output;
}
ThreeVector::~ThreeVector(){}
```

B2：概念填空题

1）在C++中，三种派生方式的说明符号为 **public** 、 **private** 、 **protected** 不加说明，则默认的派生方式为 **private** 。

2）当公有派生时，基类的公有成员成为派生类的 **公有成员** ；保护成员成为派生类的 **保护成员** ；私有成员成为派生类的 **不可直接使用的成员** 。当保护派生时，基类的公有成员成为派生类的 保护成员；保护成员成为派生类的 **保护成员** ；私有成员成为派生类的 **不可直接使用的成员** 。

3）包含成员对象的派生类的初始化分为三个步骤：首先 **调用基类构造函数** ，其次 **调用成员对象构造函数** ，最后 **调用自身构造函数** 。

4）多继承时，多个基类中的同名的成员在派生类中由于标识符不唯一而出现 **二义性** 。在派生类中采用 **基类名受限** 或 **虚基类** 来消除该问题。

B3：简答题

1）派生类如何实现对基类私有成员的访问?

- 通过基类中的非私有成员函数进行访问。
- 通过定义友元函数进行访问

2）继承与聚集之间的相同点与不同点是什么?

继承是is-a的关系，聚集是is-a-part-of的关系，继承的基类代表着子类某些公共的特征，而聚集则是部分与整体的关系。

3）什么是虚基类？它有什么作用?

在多继承中，如果直接基类有公共的基类，则会出现重复继承。这时，应把公共基类定义为相应直接基类的虚基类，来避免这个问题

B4：选择题

1）下面对派生类的描述中，错误的是（**D**）。

A．一个派生类可以作为另外一个派生类的基类

B．派生类至少有一个基类

C．派生类的成员除了它自己的成员外，还包含了它的基类的成员

D．派生类中继承的基类成员的访问权限到派生类中保持不变

2）下列对友元关系叙述正确的是（**A D**）。

A．不能继承

B．是类与类的关系

C．是一个类的成员函数与另一个类的关系

D．提高程序的运行效率

3）当保护继承时，基类的（**B**）在派生类中成为保护成员，不能通过派生类的对象来直接访问。

A．任何成员　　B．公有成员和保护成员

C．公有成员和私有成员　　D．私有成员

4）设置虚基类的目的是（**B**）。

A．简化程序　　B．消除二义性

C．提高运行效率　　D．减少目标代码

5）在公有派生情况下，有关派生类对象和基类对象的关系，不正确的叙述是（**C**）。

A．派生类的对象可以赋给基类的对象

B．派生类的对象可以初始化基类的引用

C．派生类的对象可以直接访问基类中的成员

D．派生类的对象的地址可以赋给指向基类的指针

6）有如下类定义：

```
class MyBASE
{
    int k;
public:
    void set(int n) { k = n; }
    int get() const { return k; }
};

class MyDERIVED : protected MyBASE
{
protected:
    int j;
public:
    void set(int m, int n)
    {
        MyBASE::set(m);
        j = n;
    }
    int get() const { return MyBASE::get() + j; }
};
```

则类MyDERIVED中保护成员个数是（**B**）。

A．4 B．3 C．2 D．1

7）程序如下：

```
#include <iostream>
using namespace std;
class A
{
public:
    A() { cout <<"A"; }
};

class B
```

```
{
public:
    B() { cout <<"B"; }
};

class C : public A
{
    B b;
public:
    C() { cout <<"C"; }
};

int main()
{
    C obj;
    return 0;
}
```

执行后的输出结果是（**D**）。

A. CBA　B. BAC C. ACB D. ABC

8）类O定义了私有函数F1。P和Q为O的派生类，定义为

```
class P: protected O{…};
class Q: public O{…}。
```

(**C**)可以访问F1。

A.O的对象　B.P类内 C.O类内 D.Q类内

9）有如下类定义：

```
class XA
{
    int x;
public:
    XA(int n) { x = n; }
};

class XB : public XA
{
    int y;
public:
    XB(int a, int b);
};
```

在构造函数XB的下列定义中，正确的是（**B**）。

A. XB::XB（int a，int b）：x(a)，y(b){ }

B. XB::XB（int a，int b）：XA(a)，y(b) { }

C. XB::XB（int a，int b）：x(a)，XB(b){ }

D. XB::XB（int a，int b）：XA(a)，XB(b){ }

10）有如下程序：

```
#include <iostream>
using namespace std;
class Base
{
private:
    void fun1() const { cout <<"fun1"; }

protected:
    void fun2() const { cout <<"fun2"; }

public:
    void fun3() const { cout <<"fun3"; }
};
class Derived : protected Base
{
public:
    void fun4() const { cout <<"fun4"; }
};
int main()
{
    Derived obj;
    obj.fun1(); //①
    obj.fun2(); //②
    obj.fun3(); //③
    obj.fun4(); //④
}
```

其中没有语法错误的语句是（**D**）。

A. ① B. ② C. ③ D. ④

B5：写出程序运行结果

1、

```
#include <iostream>
using namespace std;

class B1
{
public:
    B1(int i) { cout <<"constructing B1 "<< i << endl; }
    ~B1() { cout <<"destructing B1 "<< endl; }
};

class B2
{
public:
    B2() { cout <<"constructing B3 *"<< endl; }
    ~B2() { cout <<"destructing B3"<< endl; }
};

class C : public B2, virtual public B1
{
    int j;

public:
    C(int a, int b, int c) : B1(a), memberB1(b), j(c) {}

private:
    B1 memberB1;
    B2 memberB2;
};

int main()
{
    C obj(1, 2, 3);
}
```

结果为：

```
constructing B1 1
constructing B3 *
constructing B1 2
constructing B3 *
destructing B3
destructing B1
destructing B3
destructing B1
```

2、

```
#include <iostream>
using namespace std;
class B
{
public:
    void f1() { cout << "B::f1" << endl; }
};
class D : public B
{
public:
    void f1() { cout << "D::f1" << endl; }
};
void f(B &rb)
{
    rb.f1();
}
int main()
{
    D d;
    B b, &rb1 = b, &rb2 = d;
    f(rb1);
    f(rb2);
    return 0;
}
```

结果为：

```
B::f1
B::f1
```

习题九

A9

利用习题 6.9 的第 14、15 题中的时间类 Time 和日期类 Date，定义一个带日期的时间类TimeWithDate。对该类对象能进行比较、增加（增加值为秒数）、相减（结果为秒数）等操作。

```
#include <iostream>
using namespace std;
class Time
{
public:
    Time()
    {
```

```
        hour = minute = second = 0;
    }
    Time(int h, int m, int s)
    {
        hour = h;
        minute = m;
        second = s;
    }
    void set(int h, int m, int s)
    {
        hour = h;
        minute = m;
        second = s;
    }
    void increment()
    {
        if (second == 59)
        {
            if (minute == 59)
            {
                if (hour == 23)
                {
                    hour = 0;
                    minute = 0;
                    second = 0;
                }
                else
                {
                    hour++;
                    minute = 0;
                    second = 0;
                }
            }
            else
            {
                minute++;
                second = 0;
            }
        }
        else
            second++;
    }

    void display() const
    {
        cout << hour << ':' << minute << ':' << second;
    }
    bool equal(const Time &time2) const
    {
```

```
        if (hour == time2.hour && minute == time2.minute && second == time2.second)
            return true;
        else
            return false;
    }

    bool less_than(const Time &time2) const
    {
        if (hour < time2.hour || hour == time2.hour && minute < time2.minute ||
            hour == time2.hour && minute == time2.minute && second < time2.second)
            return true;
        else
            return false;
    }

protected:
    int hour;
    int minute;
    int second;
};

class Date
{
public:
    Date(int y, int m, int d)
    {
        year = y;
        month = m;
        day = d;
    }

    void increment()
    {
        int d;
        switch (month)
        {
        case 1:
        case 3:
        case 5:
        case 7:
        case 8:
        case 10:
        case 12:
            d = 31;
            break;
        case 4:
        case 6:
```

```
        case 9:
        case 11:
            d = 30;
            break;
        case 2:
            if (year % 400 == 0 || year % 4 == 0 && year % 100 != 0)
                d = 29;
            else
                d = 28;
        }
        if (day < d)
            day++;
        else if (month != 12)
        {
            day = 1;
            month++;
        }
        else
        {
            day = 1;
            month = 1;
            year++;
        }
    }

    bool equal(const Date &date2) const
    {
        return year == date2.year && month == date2.month && day == date2.day;
    }
    bool less_than(const Date &date2) const
    {
        if (year < date2.year || year == date2.year && month < date2.month ||
year == date2.year && month == date2.month && day < date2.day)
            return true;
        else
            return false;
    }

    void display() const
    {
        cout << year << '-' << month << '-' << day;
    }

protected:
    int year, month, day;
};

class TimeWithDate : public Time, public Date
{
```

```
public:
    TimeWithDate(int y, int mo, int d, int h, int mi, int s) : Date(y, mo, d), Time(h, mi, s) {}
    bool equal(const TimeWithDate &td2) const;
    bool less_than(const TimeWithDate &td2) const;
    void increment();
    int difference(const TimeWithDate &td2) const;
    void display() const;
};

bool TimeWithDate::equal(const TimeWithDate &td2) const
{
    return Time::equal(td2) && Date::equal(td2);
}
bool TimeWithDate::less_than(const TimeWithDate &td2) const
{
    if (Date::less_than(td2))
        return true;
    else if (Date::equal(td2) && Time::less_than(td2))
        return true;
    else
        return false;
}

void TimeWithDate::increment()
{
    Time::increment();
    if (((Time *)this)->equal(Time(0, 0, 0))) //或 if (hour == 0 && minute == 0 && second == 0)
        Date::increment();
}
int TimeWithDate::difference(const TimeWithDate &td2) const
{
    int days = 0;
    if (less_than(td2))
        for (TimeWithDate td = *this; td.less_than(td2); td.increment())
            days--;
    else
        for (TimeWithDate td = td2; td.less_than(*this); td.increment())
            days++;
    return days;
}

void TimeWithDate::display() const
{
    Date::display();
    cout << ' ';
    Time::display();
}
```

A2

用类模板定义一个类属的队列类。

```
template <class T, int size>
class Queue
{
    T buffer[size + 1];
    int front, rear;

public:
    Queue() { front = rear = 0; }
    bool enque(const T &x)
    {
        if ((rear + 1) % (size + 1) == front) //队列满
            return false;
        buffer[rear] = x;
        rear = (rear + 1) % (size + 1);
        return true;
    }

    bool deque(T &x)
    {
        if (front == rear) //队列空
            return false;
        x = buffer[front];
        front = (front + 1) % (size + 1);
        return true;
    }
};

```

A3

用STL的容器vector和算法sort实现习题5.8 中第16题的功能。

5.8 中第16题：编写一个程序，从键盘输入一批学生的成绩信息，每个学生的成绩信息包括：学号、姓名以及 8门课的成绩。然后按照平均成绩由高到低顺序输出学生的学号、姓名以及平均成绩。

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>
using namespace std;

class Student
{
    string id;
    string name;
```

```
    int scores[8]; //这里也可以用一个vector<int>来实现
public:
    void input_data()
    {
        cin >> id >> name;
        for (int i = 0; i < 8; i++)
            cin >> scores[i];
    }
    void display() const
    {
        cout << id << ',' << name << ',';
        for (int i = 0; i < 8; i++)
            cout << scores[i] << ',';
    }
    float average() const
    {
        int sum = 0;
        for (int i = 0; i < 8; i++)
            sum += scores[i];

        return sum / 8.0;
    }
};

bool compare_average(Student &st1, Student &st2) //比较st1和st2的平均成绩
{
    return st1.average() > st2.average();
}

void display_student_info(Student &st) //显示st的信息
{
    st.display();
    float avg_score = st.average();
    cout << avg_score;
    cout << endl;
}

int main()
{
    vector<Student> students;
    Student st;
    int t;
    for (cin >> t; t != -1; cin >> t)
    {
        st.input_data();
        students.push_back(st);
    }

    sort(students.begin(), students.end(), compare_average);
```

```
60
61      for_each(students.begin(), students.end(), display_student_info);
62
63      return 0;
64  }
```

A4

用STL的容器map和算法find_if实现例 5-11的名表表示和查找功能。

```
1   #include <iostream>
2   #include <map>
3   #include <string>
4   #include <algorithm>
5   using namespace std;
6   map<string, int> name_table;
7
8   class MatchName
9   {
10      string name;
11
12  public:
13      MatchName(string s)
14      {
15          name = s;
16      }
17      bool operator()(pair<string, int> v)
18      {
19          return v.first == name;
20      }
21  };
22
23  int main()
24  {
25      name_table["北京"] = 10;
26      name_table["上海"] = 21;
27      name_table["南京"] = 25;
28      name_table["广州"] = 20;
29      name_table["武汉"] = 27;
30      name_table["哈尔滨"] = 451;
31      name_table["西藏"] = 891;
32
33      map<string, int>::const_iterator it;                    //创建一个不能修改所指
    向的元素的迭代器
34      for (it = name_table.begin(); it != name_table.end(); it++) //遍历容器
35          cout << it->first << ": " << it->second << endl;        //输出元素的关键字和值
36
37      cout << "\n查找：南京...\n";
38      it = find_if(name_table.begin(), name_table.end(), MatchName("南京"));
```

```
    if (it != name_table.end())
        cout << it->first << ": " << it->second << endl;
    else
        cout << "未找到! \n";
    return 0;
}
```

B1：写出程序执行结果

1、

```
#include <iostream>
using namespace std;
class base
{
public:
    base() { cout << "constructing base class" << endl; }
    ~base() { cout << "destructing base class" << endl; }
};
class subs : public base
{
public:
    subs() { cout << "constructing sub class" << endl; }
    ~subs() { cout << "destructing sub class" << endl; }
};
void main()
{
    subs s;
}
```

```
constructing base class
constructing sub class
destructing sub class
destructing base class
```

2、

```
#include <iostream>
using namespace std;

class base
{
    int n;

public:
    base(int a)
    {
        cout << "constructing base class" << endl;
```

```
        n = a;
        cout << "n=" << n << endl;
    }
    ~base() { cout << "destructing base class" << endl; }
};
class subs : public base
{
    base bobj;
    int m;

public:
    subs(int a, int b, int c) : base(a), bobj(c)
    {
        cout << "constructing sub cass" << endl;
        m = b;
        cout << "m=" << m << endl;
    }
    ~subs() { cout << "destructing sub class" << endl; }
};
void main()
{
    subs s(1, 2, 3);
}
```

```
constructing base class
n=1
constructing base class
n=3
constructing sub cass
m=2
destructing sub class
destructing base class
destructing base class
```

3、

```
#include <iostream>
using namespace std;

class A
{
public:
    int n;
};
class B : public A
{
};
class C : public A
```

```
{
};
class D : public B, public C
{
    int getn() { return B::n; }
};
void main()
{
    D d;
    d.B::n = 10;
    d.C::n = 20;
    cout << d.B::n << "," << d.C::n << endl;
}
```

```
10,20
```

4、

```
#include <iostream>
using namespace std;

class base
{
    int n;

public:
    base(){};
    base(int a)
    {
        cout << "constructing base class" << endl;
        n = a;
        cout << "n=" << n << endl;
    }
    ~base()
    {
        cout << "destructing base class" << endl;
    }
};

class subs : public base
{
    int m;

public:
    subs(int a, int b) : base(a)
    {
        cout << "constructing sub class" << endl;
        m = b;
```

```
        cout << "m=" << m << endl;
    }
    ~subs()
    {
        cout << "destructing sub class" << endl;
    }
};
void main()
{
    subs s(1,2);
}
```

```
constructing base class
n=1
constructing sub class
m=2
destructing sub class
destructing base class
```

5、

```
#include <iostream>
using namespace std;
class Sample
{
protected:
    int x;

public:
    Sample() { x = 0; }
    Sample(int val) { x = val; }
    void operator++() { x++; }
};
class Derived : public Sample
{
    int y;

public:
    Derived() : Sample() { y = 0; }
    Derived(int val1, int val2) : Sample(val1) { y = val2; }
    void operator--()
    {
        x--;
        y--;
    }
    void disp()
    {
        cout << "x=" << x << " y=" << y << endl;
```

```
    }
};
void main()
{
    Derived d(3, 5);
    d.disp();
    ++d;
    d.disp();
    --d;
    --d;
    d.disp();
}
```

```
x=3 y=5
x=4 y=5
x=2 y=3
```

6、

```
#include <iostream>
using namespace std;

class A
{
    int a;

public:
    A(int i)
    {
        a = i;
        cout << "constructing class A" << endl;
    }
    void print() { cout << a << endl; }
    ~A() { cout << "destructing class A" << endl; }
};
class B1 : public A
{
    int bl;

public:
    B1(int i, int j) : A(i)
    {
        bl = j;
        cout << "constructing class B1" << endl;
    }
    void print()
    {
        A::print();
```

```
        cout << b1 << endl;
    }
    ~B1() { cout << "destructing class B1" << endl; }
};
class B2 : public A
{
    int b2;

public:
    B2(int i, int j) : A(i)
    {
        b2 = j;
        cout << "constructing class B2" << endl;
    }
    void print()
    {
        A::print();
        cout << b2 << endl;
    }
    ~B2() { cout << "destructing class B2" << endl; }
};
class C : public B1, public B2
{
    int c;

public:
    C(int i, int j, int k, int l, int m) : B1(i, j), B2(k, l), c(m)
    {
        cout << "constructing class C" << endl;
    }
    void print()
    {
        B1::print();
        B2::print();
        cout << c << endl;
    }
    ~C() { cout << "destructing class C" << endl; }
};
void main()
{
    C c1(1, 2, 3, 4, 5);
    c1.print();
}

```

```
constructing class A
constructing class B1
constructing class A
constructing class B2
```

```
constructing class C
1
2
3
4
5
destructing class C
destructing class B2
destructing class A
destructing class B1
destructing class A
```